

Department of AIML Engineering

Lab Manual

Operating System Lab

(Course Code: PCC-304-AIM)

Prepared by

Prof. Madhuri Desai



NAVSAHYADRI EDUCATION SOCIETY'S
GROUP OF INSTITUTIONS
Address: Gat. No. 69, 70, 71, Mauje Naigaon Tal: Bhor, Dist: Pune-412 205
Department of AIML Engineering



Vision and Mission of Institute:

VISION

To develop globally competent engineers for techno economic upliftment of society.

MISSION

To systematically improve educational programs, faculty capabilities, institutional infrastructure, and student competencies to meet evolving global industry standards ethically.

Vision and Mission of Department:

VISION

To be a centre of excellence in Artificial Intelligence and Machine Learning, nurturing globally competent professionals through innovation, research, entrepreneurship, and ethical technology for industry and society.

MISSION

M1. To provide **outcome-based, industry-oriented education** in AI, ML, and emerging technologies.

M2. To foster **innovation, research, problem-solving, and product development** through experiential and industry-integrated learning.

M3. To nurture **globally competent, ethical, and entrepreneurial AI professionals** creating sustainable solutions for industry and society.

PEOs and PSOs of Department:

Programme Educational Objectives (PEO'S)

Within a few years of graduation, the graduates of the AIML Engineering program will be able to:

PEO 1 – Professional Competence

Graduates will apply principles of Artificial Intelligence, Machine Learning, Computer Science, Mathematics, and Engineering to develop innovative solutions for real-world problems.

PEO 2 – Career and Higher Education

Graduates will pursue successful careers in industry, entrepreneurship, research,

and higher education in AI, ML, Data Science, and related domains.

PEO 3 – Lifelong Learning

Graduates will continuously enhance their knowledge and skills through self-learning, professional development, certifications, and emerging technologies.

PEO 4 – Ethical and Social Responsibility

Graduates will demonstrate professional ethics, social responsibility, environmental awareness, and consideration of legal aspects while developing AI-based systems.

PEO 5 – Leadership and Teamwork

Graduates will exhibit leadership qualities, effective communication, project management skills, and the ability to work collaboratively in multidisciplinary teams.

Program Specific Outcomes (PSOs)

Graduates of the AIML Engineering program will be able to:

PSO1: An ability to apply the theoretical concepts and practical knowledge of Artificial Intelligence & Machine Learning in analysis, design, development and management of information processing systems and applications in the interdisciplinary domain.

PSO2: An ability to analyze a problem, and identify and define the computing infrastructure and operations requirements appropriate to its solution. AI & ML graduates should be able to work on large-scale computing systems.

PSO3: An understanding of professional, business and business processes, ethical, legal, security and social issues and responsibilities.

PSO4: Practice communication and decision-making skills through the use of appropriate technology and be ready for professional responsibilities

PROGRAMS OUTCOMES (POs)

Engineering Graduates will be able to:

PO1 — Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals, and an engineering specialization to develop solutions for complex engineering problems.

PO2 — Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems, reaching substantiated conclusions with consideration for sustainable development.

PO3 — Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems, components, or processes to meet identified needs, with consideration for public health and safety, whole-life cost, net-zero carbon, culture, society, and environment.

PO4 — Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge, including design of experiments, modelling, and analysis and interpretation of data, to provide valid conclusions.

PO5 — Engineering Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, recognizing their limitations, to solve complex engineering problems.

PO6 — The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems, for its impact on sustainability with reference to economy, health, safety, legal framework, culture, and environment.

PO7 — Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws.

PO8 — Individual and Collaborative Team Work: Function effectively as an individual and as a member or leader in diverse/multi-disciplinary teams.

PO9 — Communication: Communicate effectively and inclusively within the engineering community and society at large — comprehend and write effective reports and design documentation, and make effective presentations, considering cultural, language, and learning differences.

PO10 — Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making, and apply these to one's own work, as a member and leader in a team, and to manage projects in multidisciplinary environments.

PO11 — Life-Long Learning: Recognize the need for, and have the preparation and ability for, (i) independent and life-long learning, (ii) adaptability to new and emerging technologies, and (iii) critical thinking in the broadest context of technological change.

Course Objective & Course Outcomes

Course Objectives

1. To introduce the fundamental structure and functions of an operating system, including process, memory, file, and I/O management.
2. To develop an understanding of process scheduling, synchronization, and deadlock handling techniques.
3. To explain memory management strategies such as paging, segmentation, and virtual memory.
4. To familiarize students with file systems, disk scheduling, and protection and security mechanisms.
5. To provide hands-on exposure to system-level programming through UNIX/Linux system calls and shell scripting.

Course Outcomes:

1. Explain the role, structure, and services of an operating system and its interaction with hardware.
2. Analyze CPU scheduling algorithms and evaluate their performance on given workloads.
3. Apply synchronization primitives and deadlock-avoidance techniques to concurrent-process problems.
4. Compare memory management and virtual memory schemes, including page replacement policies.
5. Design and implement solutions using process, thread, file, and IPC system calls in a Linux environment.

CO-PO Mapping Matrix

COs \ POs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	2	1	-	2	1	-	-	2	-	2
CO2	3	3	2	2	3	1	-	2	1	1	2
CO3	3	3	3	2	3	-	1	3	2	1	2
CO4	3	3	2	2	2	-	-	1	1	-	2
CO5	3	2	3	2	3	2	2	3	3	2	3

CO–PSO Mapping Matrix

COs \ PSOs	PSO1	PSO2	PSO3	PSO4
CO1	2	2	1	1
CO2	2	3	1	1
CO3	2	3	2	2
CO4	2	3	1	1
CO5	2	3	2	3

CO–PEO Mapping Matrix

COs \ PEOs	PEO1	PEO2	PEO3	PEO4	PEO5
CO1	2	1	2	1	1
CO2	3	2	2	1	1
CO3	3	2	2	2	3
CO4	3	2	2	1	1
CO5	3	3	3	2	3

Operating System Lab

(Course Code: PCC-304-AIM)

Teaching Scheme: Practicals: 2 Hrs/week

Examination Scheme: PR: 25 Marks & TW:25 Marks

List of Assignments / Experiments

1 Linux Environment and File Management — Perform file and directory operations in Linux using command-line utilities. Demonstrate file permissions, ownership management, process monitoring, and basic system administration commands. CO1

2 Shell Scripting and System Automation — Develop shell scripts using decision-making loops, statements, functions, and file-handling commands to automate common system tasks. CO1

3 Process Creation, Scheduling, and Performance Analysis — Develop programs to create and manage processes using system calls. Implement CPU scheduling algorithms (FCFS, SJF, Round Robin) and analyze their impact on process execution, memory utilization, and overall system performance. CO2, CO4, CO5

4 Multithreading and Resource Utilization — Implement multithreaded programs and analyze CPU utilization, memory consumption, and execution time under different thread configurations. CO2, CO3, CO4

5 Inter-Process Communication and Shared Resource Management — Implement IPC using pipes, shared memory, and message queues. Analyze communication efficiency and resource utilization. CO2, CO3, CO5

6 Synchronization Mechanisms for Concurrent Systems — Develop programs using semaphores and mutex locks to solve synchronization problems such as Producer–Consumer and Readers–Writers. Evaluate the effect of synchronization on resource allocation and system performance. CO3, CO5

7 Deadlock Detection, Avoidance, and Recovery — Implement Banker's Algorithm and deadlock detection techniques. Analyze resource allocation patterns and evaluate system performance under different deadlock scenarios. CO2, CO3, CO4

8 Memory Management and Page Replacement Strategies — Simulate paging, segmentation, and virtual memory management. Implement FIFO, LRU, and Optimal page replacement algorithms and compare their effectiveness using performance metrics. CO4, CO5

9 File System and Disk Scheduling Analysis — Implement file allocation and disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN). Analyze storage utilization, access time, and system throughput. CO4, CO5

10 Containerization and Resource Monitoring for AI/ML Workloads — Deploy a simple AI/ML application inside a Docker container. Monitor CPU, memory, storage, and process utilization using Linux tools and evaluate the benefits of container-based deployment. CO1, CO2, CO5

Subject In charge
Prof. Madhuri Desai

Experiments – CO Mapping Matrix

Exp. No.	Experiment Title	CO Mapping
1	Linux Environment and File Management	CO1
2	Shell Scripting and System Automation	CO1
3	Process Creation, Scheduling, and Performance Analysis	CO2, CO4, CO5
4	Multithreading and Resource Utilization	CO2, CO3, CO4
5	Inter-Process Communication and Shared Resource Management	CO2, CO3, CO5
6	Synchronization Mechanisms for Concurrent Systems	CO3, CO5
7	Deadlock Detection, Avoidance, and Recovery	CO2, CO3, CO4
8	Memory Management and Page Replacement Strategies	CO4, CO5
9	File System and Disk Scheduling Analysis	CO4, CO5
10	Containerization and Resource Monitoring for AI/ML Workloads	CO1, CO2, CO5

EXPERIMENT NO. 1

Title: Linux Environment and File Management

Aim

Perform file and directory operations in Linux using command-line utilities. Demonstrate file permissions, ownership management, process monitoring, and basic system administration commands.

Objectives

- Understand Linux filesystem.
- Learn file management.
- Understand permissions.
- Monitor and manage processes.

Software Requirements

- Linux operating system / Ubuntu
- Bash Shell
- Terminal

Theory

Linux follows a hierarchical tree structure beginning from the root directory (/). Everything in Linux is treated as a file.

File permissions are assigned to User, Group and Others using r(4), w(2), x(1).

Every running program is a process with a unique PID. Commands such as ps, top and kill are used to monitor and manage processes.

Linux Commands with Explanation

Command	Syntax	Purpose
pwd	pwd	Display current directory
ls	ls -la	List files
mkdir	mkdir OS_Lab	Create directory
cd	cd OS_Lab	Change directory
touch	touch file1.txt	Create file

cat	cat file1.txt	View file
cp	cp file1.txt file2.txt	Copy file
mv	mv file2.txt newfile.txt	Rename file
rm	rm newfile.txt	Delete file
rmdir	rmdir OS_Lab	Remove directory
chmod	chmod 744 file1.txt	Change permissions
chown	chown owner:group file1.txt	Change ownership
ps	ps -aux	View processes
top	top	Real-time monitor
kill	kill PID	Terminate process
whoami	whoami	Current user
date	date	Show date/time
cal	cal	Calendar
clear	clear	Clear terminal
man	man ls	Help manual

Procedure

1. Open Terminal
2. mkdir OS_Lab_P1
3. cd OS_Lab_P1
4. touch file1.txt file2.txt file3.txt
5. ls -la
6. chmod 744 file1.txt
7. ls -l
8. sleep 500 &
9. ps -aux | grep sleep
10. kill PID

Conclusion

The basic Linux commands for file management, permissions, and process management were studied and executed successfully.

Viva Questions

- What is Linux filesystem?
- What is PID?
- Difference between chmod and chown?
- Explain permission 744.
- Purpose of ps and top?

EXPERIMENT NO. 2

Title: Shell Scripting and System Automation

Aim: Develop shell scripts using decision-making statements, loops, functions, and file handling commands to automate common operating system tasks.

Objectives

- To understand the basics of shell scripting and command interpretation in Linux.
- To apply decision-making statements (if, case) for conditional logic.
- To implement loops (for, while) for task repetition.
- To use functions for modular, reusable scripts.
- To perform file handling and text processing with commands like grep, awk, sed.
- To automate common system administration tasks using shell scripts.

Software Requirements

- Linux operating system / Ubuntu
- Bash Shell
- Terminal

Theory

Shell scripting is a method of writing a sequence of commands in a script file to automate operating system tasks. Bash provides decision-making statements, loops, functions, and file handling commands for automation.

1. Decision-Making Statements

1.1 if-else – Check Even or Odd

```
#!/bin/bash
echo "Enter a number:"
read n
if [ $((n % 2)) -eq 0 ]
then
  echo "Number is Even"
else
  echo "Number is Odd"
fi
```

1.2 Nested if-else – Check Positive/Negative/Zero

```
#!/bin/bash
echo "Enter a number:"
read n
if [ $n -ge 0 ]
then
  if [ $n -eq 0 ]
  then
    echo "Number is Zero"
  else
    echo "Number is Positive"
  fi
else
  echo "Number is Negative"
fi
```

1.3 case – Simple Calculator

```
#!/bin/bash
echo "Enter two numbers:"
read a b
echo "Enter choice:"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
read ch
case $ch in
  1) echo "Addition = $((a+b))";;
  2) echo "Subtraction = $((a-b))";;
  3) echo "Multiplication = $((a*b))";;
  4) echo "Division = $((a/b))";;
  *) echo "Invalid choice";;
esac
```

2. Loop Statements

2.1 while Loop – Sum of First N Numbers

```
#!/bin/bash
echo "Enter a number:"
read n
i=1
sum=0
while [ $i -le $n ]
do
  sum=$((sum+i))
  i=$((i+1))
done
echo "Sum = $sum"
```

2.2 for Loop – Print Numbers from 1 to 10

```
#!/bin/bash
for i in {1..10}
do
  echo $i
Done
```

2.3 until Loop – Print Numbers from 1 to 5

```
#!/bin/bash
i=1
until [ $i -gt 5 ]
do
  echo $i
  i=$((i+1))
Done
```

2.4 break Statement – Stop Loop at 5

```
#!/bin/bash
for i in {1..10}
do
  if [ $i -eq 5 ]
  then
    break
  fi
  echo $i
Done
```

2.5 continue Statement – Skip Number 5

```
#!/bin/bash
for i in {1..10}
do
  if [ $i -eq 5 ]
  then
    continue
  fi
  echo $i
Done
```

3. Functions

3.1 Function – Addition of Two Numbers

```
#!/bin/bash
addition()
{
  sum=$((a+b))
  echo "Addition = $sum"
}
echo "Enter two numbers:"
read a b
Addition
```

3.2 Function with Argument – Find Square

```
#!/bin/bash
square()
{
  echo "Square = $(($1 * $1))"
}
echo "Enter a number:"
read n
square $n
```

4. File Handling

4.1 Create and Write into a File

```
#!/bin/bash
echo "Enter file name:"
read file
touch $file
echo "Enter some text:"
read text
echo "$text" > $file
echo "File created successfully."
cat $file
```

4.2 Read a File

```
#!/bin/bash
echo "Enter file name:"
read file
if [ -f $file ]
then
  echo "File contents:"
  cat $file
else
  echo "File does not exist."
Fi
```

5. Important File Handling Commands

Command	Purpose	Example
touch	Create a file	touch file.txt
cat	Display file contents	cat file.txt
cp	Copy a file	cp file.txt backup.txt
mv	Move/rename a file	mv file.txt new.txt
rm	Delete a file	rm file.txt
head	Display first lines	head file.txt
tail	Display last lines	tail file.txt
wc	Count lines, words and characters	wc file.txt

6. Procedure

11. Open the Linux Terminal.
12. Create a shell script using a text editor, for example: nano practical2.sh
13. Write the required shell commands and save the file.
14. Give execute permission using: chmod +x practical2.sh
15. Run the script using: ./practical2.sh
16. Observe and verify the output.
17. Repeat the process for decision-making, loops, functions, and file handling examples.

Conclusion

The practical demonstrates how Bash shell scripting can be used to automate common operating system tasks using control structures, reusable functions, and file operations.

Viva Questions

- What is a shell script, and what is the purpose of the #!/bin/bash line?
- Explain the if-else statement in shell scripting with an example.
- Differentiate between the for and while loops in shell scripting.
- How do you define and call a function in a shell script?
- Write a shell script to check whether a given file exists and display its contents.

EXPERIMENT NO. 3

Title:Process Creation, CPU Scheduling and Performance Analysis

Aim

To develop programs using system calls for process creation and management, and to implement CPU scheduling algorithms such as FCFS, SJF, and Round Robin.

Objectives

- To understand process creation using Linux system calls.
- To understand parent and child processes and process IDs.
- To implement FCFS, SJF, and Round Robin CPU scheduling algorithms.
- To calculate waiting time and turnaround time.
- To compare scheduling algorithms based on performance.

Software Requirements

- Ubuntu Linux / WSL
- VS Code
- GCC Compiler
- C Programming Language

Theory

A process is a program in execution. Linux provides system calls such as fork() for creating a new process. When fork() is called, the operating system creates a new process (the child) that is an almost exact copy of the calling process (the parent); both continue executing from the point of the fork() call, but fork() returns a different value to each: 0 to the child, and the child's process ID (PID) to the parent.

CPU scheduling determines which process gets the CPU and for how long:

- FCFS (First Come First Serve): Executes processes strictly in their arrival order. Simple and fair in terms of order, but can cause long waiting times for short processes stuck behind long ones (the convoy effect).
- SJF (Shortest Job First): Selects the process with the shortest burst time next. Minimizes average waiting time among non-preemptive algorithms, but requires knowing burst times in advance and can starve longer processes.
- Round Robin: Assigns each process a fixed time quantum in cyclic order, preempting a process once its quantum expires and moving it to the back of the ready queue. Provides fair CPU sharing and good response time, but its performance depends heavily on the chosen time quantum.

Program 1: Process Creation using fork()

To create a child process using the fork() system call and display process IDs.

Program code

```
#include <stdio.h>
#include <unistd.h>
int main()
{
   int pid;
   pid = fork();

   if (pid < 0)
   {
      printf("Process creation failed\n");
   }
   else if (pid == 0)
   {
      printf("Child Process\n");
      printf("Child PID = %d\n", getpid());
   }
   else
   {
      printf("Parent Process\n");
      printf("Parent PID = %d\n", getpid());
      printf("Child PID = %d\n", pid);
   }
   return 0;
}
```

Compile: gcc process.c -o process

Run: ./process

Output

```
Parent Process
Parent PID = 2450
Child PID = 2451
Child Process
Child PID = 2451
```

Program 2: FCFS CPU Scheduling

To implement First Come First Serve CPU scheduling and calculate waiting time and turnaround time.

Program code

```
#include <stdio.h>
int main()
{
   int n, i, bt[20], wt[20], tat[20];
   float awt = 0, atat = 0;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for(i = 0; i < n; i++)
   {
      printf("Enter burst time for P%d: ", i + 1);
      scanf("%d", &bt[i]);
   }
   wt[0] = 0;
   for(i = 1; i < n; i++)
     wt[i] = wt[i - 1] + bt[i - 1];
   for(i = 0; i < n; i++)
   {
      tat[i] = wt[i] + bt[i];
      awt += wt[i];
      atat += tat[i];
   }
   printf("\nProcess\tBT\tWT\tTAT\n");
   for(i = 0; i < n; i++)
      printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
   printf("\nAverage Waiting Time = %.2f\n", awt / n);
   printf("Average Turnaround Time = %.2f\n", atat / n);
   return 0;
}
```

Compile: gcc fcfs.c -o fcfs

Run: ./fcfs

Sample Input: 4 processes with burst times 5, 3, 8, 6.

Output

Process	**BT**	**WT**	**TAT**
P1	5	0	5
P2	3	5	8
P3	8	8	16
P4	6	16	22

Program 3: SJF CPU Scheduling

To implement non-preemptive Shortest Job First CPU scheduling.

Program code

```
#include <stdio.h>
int main()
{
   int n, i, j, temp;
   int bt[20], p[20], wt[20], tat[20];
   float awt = 0, atat = 0;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for(i = 0; i < n; i++)
   {
      p[i] = i + 1;
      printf("Enter burst time for P%d: ", i + 1);
      scanf("%d", &bt[i]);
   }

   for(i = 0; i < n - 1; i++)
   {
      for(j = i + 1; j < n; j++)
      {
         if(bt[i] > bt[j])
         {
            temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            temp = p[i]; p[i] = p[j]; p[j] = temp;
         }
      }
   }
   wt[0] = 0;
   for(i = 1; i < n; i++)
      wt[i] = wt[i - 1] + bt[i - 1];
   for(i = 0; i < n; i++)
   {
      tat[i] = wt[i] + bt[i];
      awt += wt[i];
      atat += tat[i];
   }
   printf("\nProcess\tBT\tWT\tTAT\n");
   for(i = 0; i < n; i++)
      printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
   printf("\nAverage Waiting Time = %.2f\n", awt / n);
   printf("Average Turnaround Time = %.2f\n", atat / n);
```

```
    return 0;
}
```

Compile: gcc sjf.c -o sjf

Run: ./sjf

Output

```
Enter number of processes: 4
Enter burst time for P1: 6
Enter burst time for P2: 8
Enter burst time for P3: 7
Enter burst time for P4: 3

Process         BT      WT      TAT
P4      3       0       3
P1      6       3       9
P3      7       9       16
P2      8       16      24

Average Waiting Time = 7.00
Average Turnaround Time = 13.00
```

Program 4: Round Robin CPU Scheduling

To implement Round Robin CPU scheduling using a time quantum.

Program code

```
#include <stdio.h>
int main()
{
    int n, i, tq, time = 0, done;
    int bt[20], rem[20], wt[20], tat[20];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("Enter burst time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem[i] = bt[i];
        wt[i] = 0;
    }
    printf("Enter time quantum: ");
    scanf("%d", &tq);
    do
    {
```

```
        done = 1;
        for(i = 0; i < n; i++)
        {
            if(rem[i] > 0)
            {
                done = 0;
                if(rem[i] > tq)
                {
                    time += tq;
                    rem[i] -= tq;
                }
                else
                {
                    time += rem[i];
                    wt[i] = time - bt[i];
                    rem[i] = 0;
                }
            }
        }
    } while(done == 0);
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
    {
        tat[i] = bt[i] + wt[i];
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
    return 0;
}
```

Compile: gcc rr.c -o rr

Run: ./rr

Sample Input: 4 processes, burst times 5, 3, 8, 6 and time quantum 2.

Output

Process	BT	WT	TAT
P1	5	12	17
P2	3	8	11
P3	8	14	22
P4	6	13	19

Important Formulas

Parameter	Formula
Waiting Time	Turnaround Time - Burst Time
Turnaround Time	Completion Time - Arrival Time
Average Waiting Time	Total Waiting Time / Number of Processes
Average Turnaround Time	Total Turnaround Time / Number of Processes

Conclusion

The process creation program was executed successfully using the fork() system call, demonstrating how a parent and child process share code but execute independently with distinct process IDs. FCFS, SJF, and Round Robin CPU scheduling algorithms were implemented successfully, and waiting time and turnaround time were calculated for performance analysis. SJF gave the lowest average waiting time for the given burst times, FCFS was simplest but suffered from the convoy effect, and Round Robin provided fair, time-sliced execution suitable for interactive systems.

Viva Questions

1. What is a process? Explain the role of the fork() system call in process creation.
2. Differentiate between the parent process and the child process in Linux, and explain how process IDs are obtained using getpid().
3. Explain the First Come First Serve (FCFS) CPU scheduling algorithm with its advantages and disadvantages.
4. Differentiate between SJF and Round Robin CPU scheduling algorithms.
5. Write a short note on the performance comparison of FCFS, SJF, and Round Robin scheduling algorithms.

EXPERIMENT NO. 4

Title: Multithreading and Resource Utilization

Aim

To implement multithreaded programs using POSIX threads (pthreads) and analyze CPU utilization, memory consumption, and execution time under different thread configurations.

Objectives

- To understand the concept of threads and multithreading in Linux.
- To understand thread creation using the pthread_create() library function.
- To differentiate between threads and processes.
- To measure execution time and resource utilization for single-threaded and multi-threaded programs.
- To analyze the effect of thread count on CPU utilization, memory consumption, and performance.

Software Requirements

- Ubuntu Linux / WSL
- VS Code
- GCC Compiler
- C Programming Language
- pthread library
- Linux monitoring tools (top, htop, time, ps)

Theory

A thread is the smallest unit of execution within a process. Multiple threads of the same process share the same address space, code segment, and open files, but each has its own stack, program counter, and register set.

Unlike processes, threads share memory, which makes communication between them faster but requires synchronization to avoid race conditions.

POSIX threads (pthreads) is the standard threading API in Linux. A thread is created using pthread_create() and joined using pthread_join().

Multithreading improves CPU utilization by allowing parallel execution on multi-core systems, reducing overall execution time for CPU-intensive tasks. The number of threads and their configuration directly affect resource utilization.

Program 1: Thread Creation using pthread_create()

To create multiple threads and display their thread IDs and execution.

Program code

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
void *thread_func(void *arg)
{
   int id = *(int *)arg;
   printf("Thread %d running\n", id);
   printf("Thread %d ID = %lu\n", id, pthread_self());
   return NULL;
}
int main()
{
   pthread_t t1, t2;
   int a = 1, b = 2;
   pthread_create(&t1, NULL, thread_func, &a);
   pthread_create(&t2, NULL, thread_func, &b);
   pthread_join(t1, NULL);
   pthread_join(t2, NULL);
   printf("Main thread finished\n");
   return 0;
}
```

Compile: gcc thread.c -o thread -lpthread

Run: ./thread

Output

```
Thread 1 running
Thread 1 ID = 140234567890112
Thread 2 running
Thread 2 ID = 140234559497408
Main thread finished
```

Program 2: Sequential (Single-Threaded) vs Multithreaded Execution Time

To compare execution time of a CPU-intensive task in single-threaded and multi-threaded modes.

Program code

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
#define N 100000000
void *work(void *arg)
{
   volatile double sum = 0;
   for(long i = 0; i < N / 2; i++)
      sum += i * 0.5;
   return NULL;
}
int main()
{
   clock_t start, end;
   pthread_t t1, t2;
   /* Sequential */
   start = clock();
   work(NULL);
   work(NULL);
   end = clock();
   printf("Sequential time = %.4f sec\n",
         (double)(end - start) / CLOCKS_PER_SEC);
   /* Multithreaded */
   start = clock();
   pthread_create(&t1, NULL, work, NULL);
   pthread_create(&t2, NULL, work, NULL);
   pthread_join(t1, NULL);
   pthread_join(t2, NULL);
   end = clock();
   printf("Multithreaded time = %.4f sec\n",
         (double)(end - start) / CLOCKS_PER_SEC);
   return 0;
}
```

Compile: gcc compare.c -o compare -lpthread

Run: ./compare

Output

```
Sequential time = 0.5820 sec
Multithreaded time = 0.3050 sec
```

Program 3: Resource Monitoring under Different Thread Configurations

To monitor CPU and memory utilization for different thread counts using Linux tools.

Program Code

```
#include <stdio.h>
#include <pthread.h>
#define THREADS 4
#define N 50000000
void *load(void *arg)
{
   volatile double x = 0;
   for(long i = 0; i < N; i++)
      x += i * 0.5;
   return NULL;
}
int main()
{
   pthread_t t[THREADS];
   int i;
   for(i = 0; i < THREADS; i++)
      pthread_create(&t[i], NULL, load, NULL);

   for(i = 0; i < THREADS; i++)
      pthread_join(t[i], NULL);

   printf("%d threads completed\n", THREADS);
   return 0;
}
```

Compile: gcc monitor.c -o monitor -lpthread

Run: ./monitor (in one terminal) and top -H or time ./monitor (in another)

Observation

Threads	CPU Utilization	Memory (RSS)	Execution Time
1	~25%	~1.5 MB	0.98 s
2	~50%	~1.6 MB	0.51 s
4	~95%	~1.8 MB	0.28 s

Important Formulas

Parameter	Formula
Speedup	Sequential Time / Multithreaded Time
Efficiency	Speedup / Number of Threads
CPU Utilization	(Busy Time / Total Time) x 100
Memory per Thread	Total Memory / Number of Threads

Conclusion

Thread creation programs were executed successfully using the pthread_create() library. Execution time, CPU utilization, and memory consumption were measured under different thread configurations and compared for performance analysis.

Viva Questions

- What is a thread? Differentiate between a thread and a process.
- Explain the role of pthread_create() and pthread_join() in multithreaded programming.
- What is the advantage of multithreading over sequential execution in terms of CPU utilization?
- How does the number of threads affect CPU utilization, memory consumption, and execution time?
- Define speedup and efficiency in the context of multithreaded performance analysis. State their formulae.

EXPERIMENT NO. 5

Title: Inter-Process Communication and Shared Resource Management

Aim

To implement Inter-Process Communication (IPC) using pipes, shared memory, and message queues, and analyze communication efficiency and resource utilization.

Objectives

- To understand IPC mechanisms in Linux.
- To implement unnamed pipes for parent-child communication.
- To use shared memory for data sharing.
- To implement message queues for process communication.
- To analyze communication overhead and resource utilization.

Software Requirements

- Ubuntu Linux / WSL
- GCC Compiler
- VS Code
- Linux IPC utilities
- C Programming Language

Theory

Inter-Process Communication (IPC) enables processes to exchange information and synchronize activities. Since processes normally execute in their own isolated address spaces, the operating system provides specific mechanisms that allow controlled communication and data sharing between them.

Types of IPC:

- Pipes
- Shared Memory
- Message Queues
- Signals
- Sockets

Shared memory provides the fastest communication because processes access the same memory area directly, without the kernel copying data between address spaces on every operation.
Message queues allow structured communication using messages, where each message has a type and is stored in a kernel-maintained queue until the receiving process retrieves it.

Program 1: IPC using Pipe

To establish one-way communication between a parent and child process using an unnamed pipe.

Program code

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main()
{
   int fd[2];
   char msg[] = "Hello from Parent";
   char buffer[100];

   pipe(fd);

   if(fork() == 0)
   {
      read(fd[0], buffer, sizeof(buffer));
      printf("Child received: %s\n", buffer);
   }
   else
   {
      write(fd[1], msg, strlen(msg) + 1);
   }

   return 0;
}
```

Compile: gcc pipe.c -o pipe_ipc

Run: ./pipe_ipc

Output

```
Child received: Hello from Parent
```

Program 2: Shared Memory

To create a shared memory segment and write data into it that can be accessed by other processes attaching to the same segment.

Program code

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

```
#include <string.h>

int main()
{
   int shmid;
   char *str;

   shmid = shmget(1234, 1024, 0666 | IPC_CREAT);

   str = (char*) shmat(shmid, NULL, 0);

   strcpy(str, "Shared Memory Example");

   printf("Data written: %s\n", str);

   shmdt(str);

   return 0;
}
```

Compile: gcc shmem.c -o shmem

Run: ./shmem

Output

```
Data written: Shared Memory Example
```

Note: The shared memory segment created with key 1234 persists in the kernel until explicitly removed. Use ipcs -m to list active shared memory segments and ipcrm -m <shmid> to remove one after use.

Program 3: Message Queue

To send a structured message to a System V message queue, which another process can later retrieve using msgrcv().

Program code

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct msg
{
   long type;
   char text[100];
};
```

```
int main()
{
   key_t key = ftok("prog", 65);

   int msgid = msgget(key, 0666 | IPC_CREAT);

   struct msg message;

   message.type = 1;

   sprintf(message.text, "Hello Queue");

   msgsnd(msgid, &message, sizeof(message), 0);

   printf("Message sent\n");

   return 0;
}
```

Compile: gcc msgqueue.c -o msgqueue

Run: ./msgqueue

Output

```
Message sent
```

Note: ftok("prog", 65) requires a file named prog to exist in the working directory to generate a unique key; use ipcs -q to view active message queues and ipcrm -q <msgid> to remove one.

Observation Table

IPC Method	Communication Speed	Memory Usage
Pipe	Medium	Low
Shared Memory	High	Medium
Message Queue	Medium	Medium

Formula

Communication Efficiency = Data Transferred / Communication Time

Conclusion

IPC mechanisms were implemented successfully using pipes, shared memory, and message queues. The pipe example demonstrated simple one-way parent-child communication, shared

memory showed the fastest data exchange since processes access a common memory region directly without kernel-mediated copying, and message queues provided structured, type-based communication suitable for more complex inter-process messaging. Overall, shared memory offers the best raw communication speed but requires explicit synchronization, while pipes and message queues offer simpler, kernel-managed communication at a moderate performance cost.

Viva Questions

1. What is IPC?
2. Differentiate pipe and shared memory.
3. What are message queues?
4. Why is shared memory faster?
5. Explain IPC resource utilization.

EXPERIMENT NO. 6

Title: Synchronization Mechanisms for Concurrent Systems

Aim

To develop programs using semaphores and mutex locks to solve synchronization problems such as Producer-Consumer and Readers-Writers.

Objectives

- Understand synchronization concepts.
- Implement mutex locks.
- Implement semaphores.
- Solve the Producer-Consumer problem.
- Solve the Readers-Writers problem.
- Analyze synchronization overhead.

Software Requirements

- Ubuntu Linux
- GCC Compiler
- POSIX Threads Library
- Semaphore Library

Theory

Synchronization prevents race conditions when multiple processes or threads access shared resources simultaneously. Without proper synchronization, the outcome of concurrent execution can depend on the timing of thread scheduling, leading to inconsistent or incorrect results.

Synchronization Mechanisms:

- Mutex: A mutual exclusion lock that allows only one thread to enter the critical section at a time. A thread must lock the mutex before entering and unlock it after leaving.
- Semaphore: An integer variable accessed only through the atomic operations wait() (P) and signal() (V). A binary semaphore behaves like a mutex, while a counting semaphore can allow a limited number of threads to access a resource pool.
- Spin Lock: A lock where a thread repeatedly checks ("busy-waits") until the lock becomes available, avoiding the overhead of putting the thread to sleep, useful for very short critical sections.
- Condition Variable: Allows threads to wait until a particular condition becomes true, and is typically used together with a mutex to avoid missed signals.

Race Condition: A race condition occurs when the outcome of a program depends on the relative timing of uncontrolled events, such as two threads accessing and modifying shared data at the same time without synchronization.

Program 1: Mutex Example

To protect a critical section using a POSIX mutex lock so that only one thread executes it at a time.

Program code

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t lock;

void *task()
{
   pthread_mutex_lock(&lock);

   printf("Critical Section\n");

   pthread_mutex_unlock(&lock);

   return NULL;
}

int main()
{
   pthread_t t1, t2;

   pthread_mutex_init(&lock, NULL);

   pthread_create(&t1, NULL, task, NULL);
   pthread_create(&t2, NULL, task, NULL);

   pthread_join(t1, NULL);
   pthread_join(t2, NULL);

   pthread_mutex_destroy(&lock);
   return 0;
}
```

Compile: gcc mutex.c -o mutex -lpthread

Run: ./mutex

Output

```
Critical Section
Critical Section
```

Program 2: Producer-Consumer Problem using Semaphores

To synchronize a producer thread and a consumer thread accessing a shared bounded buffer, using counting semaphores for empty/full slots and a mutex-style semaphore for mutual exclusion.

Program code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full, mutex;

void *producer(void *arg)
{
   for(int i = 1; i <= ITEMS; i++)
   {
      sem_wait(&empty);
      sem_wait(&mutex);

      buffer[in] = i;
      printf("Producer produced: %d at slot %d\n", i, in);
      in = (in + 1) % BUFFER_SIZE;

      sem_post(&mutex);
      sem_post(&full);
   }
   return NULL;
}

void *consumer(void *arg)
{
   for(int i = 1; i <= ITEMS; i++)
   {
      sem_wait(&full);
      sem_wait(&mutex);

      int item = buffer[out];
      printf("Consumer consumed: %d from slot %d\n", item, out);
      out = (out + 1) % BUFFER_SIZE;
```

```
        sem_post(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main()
{
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);
    return 0;
}
```

Compile: gcc prodcons.c -o prodcons -lpthread

Run: ./prodcons

Output

```
Producer produced: 1 at slot 0
Producer produced: 2 at slot 1
Consumer consumed: 1 from slot 0
Producer produced: 3 at slot 2
Consumer consumed: 2 from slot 1
...
Producer produced: 10 at slot 4
Consumer consumed: 10 from slot 4
```

Note: The empty semaphore (initialized to BUFFER_SIZE) tracks free slots, full (initialized to 0) tracks filled slots, and mutex ensures only one thread updates the buffer index at a time. The exact interleaving of "produced"/"consumed" lines can vary between runs since both threads run concurrently, but the buffer is never overrun or under-read.

Program 3: Readers-Writers Problem using Semaphore and Mutex

To allow multiple reader threads to read shared data concurrently, while ensuring a writer thread gets exclusive access (no readers or other writers) when writing.

Program code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int sharedData = 0;
int readCount = 0;

sem_t writeLock;
pthread_mutex_t readCountLock;

void *reader(void *arg)
{
   int id = *(int *)arg;

   pthread_mutex_lock(&readCountLock);
   readCount++;
   if(readCount == 1)
      sem_wait(&writeLock);   /* first reader blocks writers */
   pthread_mutex_unlock(&readCountLock);

   printf("Reader %d read value: %d\n", id, sharedData);
   usleep(1000);

   pthread_mutex_lock(&readCountLock);
   readCount--;
   if(readCount == 0)
      sem_post(&writeLock);  /* last reader unblocks writers */
   pthread_mutex_unlock(&readCountLock);

   return NULL;
}

void *writer(void *arg)
{
   int id = *(int *)arg;

   sem_wait(&writeLock);
   sharedData += 10;
```

```
    printf("Writer %d wrote value: %d\n", id, sharedData);
    sem_post(&writeLock);

    return NULL;
}

int main()
{
    pthread_t r[3], w[2];
    int rid[3] = {1, 2, 3}, wid[2] = {1, 2};

    sem_init(&writeLock, 0, 1);
    pthread_mutex_init(&readCountLock, NULL);

    pthread_create(&w[0], NULL, writer, &wid[0]);
    pthread_create(&r[0], NULL, reader, &rid[0]);
    pthread_create(&r[1], NULL, reader, &rid[1]);
    pthread_create(&w[1], NULL, writer, &wid[1]);
    pthread_create(&r[2], NULL, reader, &rid[2]);

    pthread_join(w[0], NULL);
    pthread_join(r[0], NULL);
    pthread_join(r[1], NULL);
    pthread_join(w[1], NULL);
    pthread_join(r[2], NULL);

    sem_destroy(&writeLock);
    pthread_mutex_destroy(&readCountLock);
    return 0;
}
```

Compile: gcc readwrite.c -o readwrite -lpthread

Run: ./readwrite

Output

```
Writer 1 wrote value: 10
Reader 1 read value: 10
Reader 2 read value: 10
Writer 2 wrote value: 20
Reader 3 read value: 20
```

Observation

Method	Protection	Speed
Mutex	High	High
Semaphore	High	Medium

Formula

CPU Utilization = (Busy Time / Total Time) x 100

Conclusion

Synchronization mechanisms successfully prevented race conditions. The mutex example ensured mutually exclusive access to the critical section, the Producer-Consumer problem was solved using counting semaphores to safely coordinate a shared bounded buffer between concurrent threads, and the Readers-Writers problem was solved using a combination of a semaphore and a mutex to allow concurrent reads while giving writers exclusive access. Semaphores offer more flexibility than a plain mutex, at a small additional coordination overhead.

Viva Questions

1. What is synchronization?
2. Difference between semaphore and mutex.
3. Explain the Producer-Consumer problem.
4. What is a race condition?
5. Why is synchronization necessary?

EXPERIMENT NO. 7

Title: Deadlock Detection, Avoidance, and Recovery

Aim To implement the Banker's Algorithm for deadlock avoidance and resource-request handling, implement deadlock detection techniques, analyze resource allocation patterns, and evaluate system performance under different deadlock scenarios

Objectives

- To understand the core concepts of deadlocks, including the four necessary conditions (Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait).To implement the Banker's Algorithm to determine safe sequences and evaluate resource request safety.
- To implement a deadlock detection algorithm using wait-for graphs or allocation matrices.
- To analyze resource allocation patterns and simulate recovery techniques (process termination or resource preemption).
- To evaluate system performance and resource utilization efficiency across safe, unsafe, and deadlocked environments.

Software Requirements

- Ubuntu Linux / WSL
- VS Code
- GCC Compiler
- C Programming Language

Theory

A deadlock occurs in a multi-programming environment when two or more processes are unable to proceed because each is waiting for a resource held by another process in the cycle.

To handle deadlocks, operating systems use two major strategy pillars:

· Deadlock Avoidance (Banker's Algorithm): A proactive method that ensures the system never enters an unsafe state. The system dynamically monitors resource allocation status using vectors (Available) and matrices (Max, Allocation, and Need, where Need =Max−Allocation). A resource request is granted only if the resulting state remains safe, meaning there exists a sequence in which all processes can execute completely without deadlocking.

· Deadlock Detection and Recovery: A reactive method where the system lets requests proceed unchecked, periodically running a detection algorithm (checking for cycles via resource allocation or wait-for graphs). If a deadlock is verified, it triggers recovery actions like process termination (aborting all or one by one) or resource preemption (taking resources away from a process).

Program 1: Deadlock Avoidance using Banker's Algorithm

Implementation of the matrix safety tracking algorithm to compute valid execution vectors for multi-resource allocations

```
#include <stdio.h>

#include <stdbool.h>

#define P 5
#define R 3
void calculateNeed(int need[P][R], int max[P][R], int alloc[P][R]) {
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max[i][j] - alloc[i][j];
}
bool isSafe(int processes[], int avail[], int max[][R], int alloc[][R]) {
    int need[P][R];
    calculateNeed(need, max, alloc);
    bool finish[P] = {false};
    int safeSeq[P];
    int work[R];
    for (int i = 0; i < R; i++) work[i] = avail[i];
    int count = 0;
    while (count < P) {
        bool found = false;
        for (int p = 0; p < P; p++) {
            if (!finish[p]) {
                int j;
                for (j = 0; j < R; j++)
                    if (need[p][j] > work[j]) break;
                if (j == R) {
                    for (int k = 0; k < R ; k++) work[k] += alloc[p][k];
                    safeSeq[count++] = p;
                    finish[p] = true;
                    found = true;
                }
            }
        }
        if (!found) {
            printf("System is NOT in a safe state!\n");
            return false;
        }
    }
    printf("System is in a SAFE state.\nSafe sequence: ");
    for (int i = 0; i < P; i++) printf("P%d ", safeSeq[i]);
    printf("\n");
    return true;
}
int main() {
    int processes[] = {0, 1, 2, 3, 4};
    int avail[] = {3, 3, 2};
    int max[P][R] = {{7,5,3}, {3,2,2}, {9,0,2}, {2,2,2}, {4,3,3}};
    int alloc[P][R] = {{0,1,0}, {2,0,0}, {3,0,2}, {2,1,1}, {0,0,2}};
    isSafe(processes, avail, max, alloc);
    return 0;
}
```

Compile: gcc banker.c -o banker **Run:** ./banker

Output:
System is in a SAFE state.
Safe sequence: P1 P3 P4 P0 P2

Program 2: Deadlock Detection and Recovery Simulation

Implementation of multi-resource trace matrix scanning to detect active circular locks and simulate process victim termination.

```
#include <stdio.h>
#include <stdbool.h>
#define P 3
#define R 3
int main() {
    int avail[] = {0, 0, 0};
    int alloc[P][R] = {{0,1,0}, {0,0,1}, {1,0,0}};
    int request[P][R] = {{0,0,1}, {1,0,0}, {0,1,0}};
    int work[R];
    for (int i = 0; i < R; i++) work[i] = avail[i];
    bool finish[P] = {false};
    for (int i = 0; i < P; i++) {
        int allocated = 0;
        for (int j = 0; j < R; j++) allocated += alloc[i][j];
        if (allocated == 0) finish[i] = true;
    }
    for (int k = 0; k < P; k++) {
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                int j;
                for (j = 0; j < R; j++) if (request[i][j] > work[j]) break;
                if (j == R) {
                    for (int m = 0; m < R; m++) work[m] += alloc[i][m];
                    finish[i] = true;
                }
            }
        }
    }
    bool deadlocked = false;
    for (int i = 0; i < P; i++) {
        if (!finish[i]) {
            printf("Deadlock detected! Process P%d is deadlocked.\n", i);
            deadlocked = true;
        }
    }

if (deadlocked)
{
```

```
    printf("\n[Recovery]: Aborting Process P2 to break circular wait chain...\n");
        for (int m = 0; m < R; m++) work[m] += alloc[2][m];
        finish[2] = true;
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                int j;
                for (j = 0; j < R; j++) if (request[i][j] > work[j]) break;
                if (j == R) {
                    for (int m = 0; m < R; m++) work[m] += alloc[i][m];
                    finish[i] = true;
                    printf("Process P%d cleared successfully.\n", i);
                }
            }
        }
    }
    return 0;
}
```

Compile: gcc detect.c -o detect **Run:** ./detect

Output:

Deadlock detected! Process P0 is deadlocked.
Deadlock detected! Process P1 is deadlocked.
Deadlock detected! Process P2 is deadlocked.
[Recovery]: Aborting Process P2 to break circular wait chain...
Process P0 cleared successfully.
Process P1 cleared successfully.

Observation & Performance Analysis

Scenario Configuration	Algorithmic Overhead	System Status	CPU Utilization	Memory (RSS)	Throughput
Scenario A: Balanced Vectors	0.0012 s	Safe State	~12%	~1.1 MB	High
Scenario B: Dense High-Stress	0.0045 s	Unsafe State	~45%	~1.4 MB	Moderate
Scenario C: Interlocked Cycle	0.0089 s	Deadlocked	~98%	~1.9 MB	Zero (Stalled)

Important Formulas

Parameter	Formula / Expression
Resource Need Matrix	Need[i][j] = Max[i][j] - Allocation[i][j]
Safety Vector Condition Checking	Request[i][j] <= Need[i][j] AND Request[i][j] <= Available[j]
Resource Recovery Rate (RRR)	Reclaimed Resources = Sum(Allocated Resources of Aborted Victims)
System Degradation Factor (SDF)	SDF = ((CPU_deadlock - CPU_normal) / CPU_normal) * 100

Conclusion

The Banker's Algorithm simulation successfully validated system safety states and calculated safe execution sequences to actively avoid deadlock traps. The reactive deadlock detection framework effectively isolated loop conditions within resource allocation patterns, verifying that selective process victim abortion functions properly reclaim system resources and eliminate runtime degradation.

Viva Questions

1. What are the four necessary conditions that must hold simultaneously for a structural deadlock to manifest?
2. Differentiate between an Unsafe State and a Deadlocked State clearly within multi-process environments.
3. Explain the role of the Need Matrix inside the Banker's Algorithm and how it restricts rapid vector updates.
4. What are the primary operational trade-offs encountered between implementing proactive avoidance versus reactive detection?
5. Discuss selection metrics utilized by operating system schedulers when choosing process victims for deadlock recovery.

EXPERIMENT NO. 8

Title: Memory Management and Page Replacement Strategies

Aim

To simulate paging, segmentation, and virtual memory management and compare FIFO, LRU, and Optimal page replacement algorithms.

Objectives

- To understand the concept of paging and segmentation as memory management schemes.
- To understand virtual memory and the need for page replacement when memory is full.
- To implement FIFO, LRU, and Optimal page replacement algorithms in C.
- To calculate the number of page faults and hit ratio for each algorithm.
- To compare the performance of FIFO, LRU, and Optimal algorithms for the same reference string.

Software Requirements

- Ubuntu Linux / WSL
- VS Code
- GCC Compiler
- C Programming Language

Theory

Paging: Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory. The logical address space of a process is divided into fixed-size blocks called pages, and physical memory is divided into blocks of the same size called frames. A page table maps each page to a frame, allowing a process's memory to be scattered across non-contiguous frames.

Segmentation: Segmentation divides a program into logical units such as code, data, and stack segments, each of variable size. Unlike paging, segments correspond to logical divisions of a program as seen by the programmer, and each segment is mapped independently into physical memory using a segment table containing base and limit values.

Virtual Memory: Virtual memory allows a process to execute even if it is not entirely in physical memory, by keeping only the actively used pages in RAM and the rest on disk. When a process references a page that is not currently in memory, a page fault occurs, and the operating system must bring that page into memory, replacing an existing page if memory is full.

Page Replacement Algorithms: When a page fault occurs and no free frame is available, the OS must select an existing page to evict. The choice of victim page significantly affects performance.

- FIFO (First In First Out): Replaces the page that has been in memory the longest, regardless of how recently it was used. Simple to implement but can suffer from Belady's Anomaly, where increasing the number of frames increases page faults.

- LRU (Least Recently Used): Replaces the page that has not been used for the longest time. Approximates the optimal strategy well in practice and does not suffer from Belady's Anomaly, but is costlier to implement since it requires tracking usage history.
- Optimal: Replaces the page that will not be used for the longest time in the future. It gives the lowest possible number of page faults for a given reference string, but requires future knowledge and so is used only as a theoretical benchmark for comparison.

Program 1: Paging Simulation

To simulate address translation using a page table that maps logical pages to physical frames.

Program code

```
#include <stdio.h>
#define PAGE_SIZE 1024
int main()
{
   int pageTable[10], numPages, logicalAddr, page, offset, physicalAddr;
   printf("Enter number of pages: ");
   scanf("%d", &numPages);
   for(int i = 0; i < numPages; i++)
   {
      printf("Enter frame number for page %d: ", i);
      scanf("%d", &pageTable[i]);
   }
   printf("Enter logical address: ");
   scanf("%d", &logicalAddr);

   page = logicalAddr / PAGE_SIZE;
   offset = logicalAddr % PAGE_SIZE;

   if(page >= numPages)
   {
      printf("Invalid page reference (page fault - out of range)\n");
      return 0;
   }

   physicalAddr = pageTable[page] * PAGE_SIZE + offset;
   printf("Page number = %d, Offset = %d\n", page, offset);
   printf("Physical Address = %d\n", physicalAddr);
   return 0;
}
```

Compile: gcc paging.c -o paging

Run: ./paging

Output

```
Enter number of pages: 4
Enter frame number for page 0: 3
Enter frame number for page 1: 1
Enter frame number for page 2: 4
Enter frame number for page 3: 0
Enter logical address: 2500

Page number = 2, Offset = 452
Physical Address = 4548
```

Program 2: Segmentation Simulation

To simulate address translation using a segment table with base and limit for each segment.

Program code

```
#include <stdio.h>
int main()
{
   int base[10], limit[10], numSeg, segNo, offset, physicalAddr;
   printf("Enter number of segments: ");
   scanf("%d", &numSeg);
   for(int i = 0; i < numSeg; i++)
   {
      printf("Enter base and limit for segment %d: ", i);
      scanf("%d %d", &base[i], &limit[i]);
   }
   printf("Enter segment number and offset: ");
   scanf("%d %d", &segNo, &offset);

   if(segNo >= numSeg || offset >= limit[segNo])
   {
      printf("Segmentation fault: invalid address\n");
      return 0;
   }

   physicalAddr = base[segNo] + offset;
   printf("Physical Address = %d\n", physicalAddr);
   return 0;
}
```

Compile: gcc segmentation.c -o segmentation

Run: ./segmentation

Output

```
Enter number of segments: 3
Enter base and limit for segment 0: 1000 400
Enter base and limit for segment 1: 2000 150
Enter base and limit for segment 2: 3000 600
Enter segment number and offset: 1 120

Physical Address = 2120
```

Program 3: FIFO Page Replacement

To replace the oldest page in memory when a page fault occurs and no free frame is available.

Program code

```
#include <stdio.h>
int main()
{
   int pages[20], n, frames[10], frameCount, faults = 0, i, j, found, front = 0;
   printf("Enter number of pages in reference string: ");
   scanf("%d", &n);
   printf("Enter reference string: ");
   for(i = 0; i < n; i++)
      scanf("%d", &pages[i]);
   printf("Enter number of frames: ");
   scanf("%d", &frameCount);

   int filled = 0;
   for(i = 0; i < frameCount; i++) frames[i] = -1;

   for(i = 0; i < n; i++)
   {
      found = 0;
      for(j = 0; j < frameCount; j++)
         if(frames[j] == pages[i]) { found = 1; break; }

      if(!found)
      {
         frames[front] = pages[i];
         front = (front + 1) % frameCount;
         faults++;
      }
   }
   printf("\nTotal Page Faults (FIFO) = %d\n", faults);
   printf("Hit Ratio = %.2f\n", (float)(n - faults) / n);
```

```
    return 0;
}
```

Compile: gcc fifo.c -o fifo

Run: ./fifo

Sample Input: Reference string = 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1; Frames = 3

Output

```
Enter number of pages in reference string: 20
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames: 3

Total Page Faults (FIFO) = 15
Hit Ratio = 0.25
```

Program 4: LRU Page Replacement

To replace the page that has not been used for the longest time when a page fault occurs.

Program code

```
#include <stdio.h>
int main()
{
    int pages[20], n, frames[10], recent[10], frameCount, faults = 0, i, j, found;
    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &frameCount);

    for(i = 0; i < frameCount; i++) frames[i] = -1;

    for(i = 0; i < n; i++)
    {
        found = -1;
        for(j = 0; j < frameCount; j++)
            if(frames[j] == pages[i]) { found = j; break; }

        if(found == -1)
        {
            int lruIndex = 0, minRecent = recent[0];
            for(j = 1; j < frameCount; j++)
                if(frames[j] == -1) { lruIndex = j; break; }
```

```
            else if(recent[j] < minRecent) { minRecent = recent[j]; lruIndex = j; }

         frames[lruIndex] = pages[i];
         recent[lruIndex] = i;
         faults++;
      }
      else
      {
         recent[found] = i;
      }
   }
   printf("\nTotal Page Faults (LRU) = %d\n", faults);
   printf("Hit Ratio = %.2f\n", (float)(n - faults) / n);
   return 0;
}
```

Compile: gcc lru.c -o lru

Run: ./lru

Sample Input: Same reference string as above; Frames = 3

Output

```
Enter number of pages in reference string: 20
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames: 3

Total Page Faults (LRU) = 16
Hit Ratio = 0.20
```

Program 5: Optimal Page Replacement

To replace the page that will not be used for the longest time in the future, giving the theoretical minimum number of page faults.

Program code

```
#include <stdio.h>
int main()
{
   int pages[20], n, frames[10], frameCount, faults = 0, i, j, k, found;
   printf("Enter number of pages in reference string: ");
   scanf("%d", &n);
   printf("Enter reference string: ");
   for(i = 0; i < n; i++)
      scanf("%d", &pages[i]);
   printf("Enter number of frames: ");
   scanf("%d", &frameCount);
```

```
  for(i = 0; i < frameCount; i++) frames[i] = -1;

  for(i = 0; i < n; i++)
  {
     found = 0;
     for(j = 0; j < frameCount; j++)
        if(frames[j] == pages[i]) { found = 1; break; }

     if(!found)
     {
        int emptyIndex = -1;
        for(j = 0; j < frameCount; j++)
           if(frames[j] == -1) { emptyIndex = j; break; }

        if(emptyIndex != -1)
        {
           frames[emptyIndex] = pages[i];
        }
        else
        {
           int farthest = -1, replaceIndex = 0;
           for(j = 0; j < frameCount; j++)
           {
              int nextUse = 9999;
              for(k = i + 1; k < n; k++)
                 if(pages[k] == frames[j]) { nextUse = k; break; }

              if(nextUse > farthest) { farthest = nextUse; replaceIndex = j; }
           }
           frames[replaceIndex] = pages[i];
        }
        faults++;
     }
  }
  printf("\nTotal Page Faults (Optimal) = %d\n", faults);
  printf("Hit Ratio = %.2f\n", (float)(n - faults) / n);
  return 0;
}
```

Compile: gcc optimal.c -o optimal

Run: ./optimal

Sample Input: Same reference string as above; Frames = 3

Output

```
Enter number of pages in reference string: 20
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames: 3

Total Page Faults (Optimal) = 11
Hit Ratio = 0.45
```

Observation Table (Page Replacement Comparison)

Algorithm	Total Page Faults	Hit Ratio
FIFO	15	0.25
LRU	16	0.20
Optimal	11	0.45

(Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1; Frames = 3)

Important Formulas

Parameter	Formula
Page Fault Ratio	Number of Page Faults / Total Page References
Hit Ratio	(Total References - Page Faults) / Total References
Physical Address (Paging)	(Frame Number x Page Size) + Offset
Physical Address (Segmentation)	Segment Base + Offset (valid only if Offset < Limit)

Conclusion

Paging and segmentation were simulated to understand logical-to-physical address translation, and FIFO, LRU, and Optimal page replacement algorithms were implemented and compared for the same reference string. The Optimal algorithm produced the fewest page faults as expected, since it uses future knowledge of references, while LRU performed close to Optimal by using past access history, and FIFO performed the worst since it ignores usage patterns entirely. This confirms that the choice of page replacement algorithm has a direct and measurable impact on system performance.

Viva Questions

1. What is the difference between paging and segmentation as memory management techniques?
2. What is a page fault, and under what conditions does it occur?
3. Explain the FIFO page replacement algorithm and describe Belady's Anomaly.
4. Why does LRU generally perform better than FIFO in practice?
5. Why is the Optimal page replacement algorithm not implementable in a real operating system?
6. Define hit ratio and page fault ratio. How are they calculated?

EXPERIMENT NO. 9

Title: File System and Disk Scheduling Analysis

Aim

To implement file allocation and disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN) and analyze storage utilization, access time, and system throughput.

CO Mapping

CO4, CO5

Objectives

- To understand disk scheduling and why it's needed in operating systems.
- To implement FCFS, SSTF, SCAN, and C-SCAN disk scheduling algorithms in C.
- To calculate total head movement and average seek time for each algorithm.
- To understand file allocation methods (contiguous, linked, indexed) and their effect on storage utilization.
- To compare scheduling algorithms based on access time, throughput, and efficiency.

Software Requirements

- Ubuntu Linux / WSL
- VS Code
- GCC Compiler
- C Programming Language

Theory

Disk Scheduling: When multiple I/O requests arrive for a disk, the operating system must decide the order in which to service them. Since disk access involves mechanical movement of the read/write head, the order directly affects seek time, and therefore overall system performance.

Seek Time: The time taken to move the disk head from its current track to the target track, proportional to the number of tracks moved (head movement).

Common Algorithms:

- FCFS (First Come First Served): Requests are serviced strictly in the order they arrive. Simple but can cause large, inefficient head movements.
- SSTF (Shortest Seek Time First): Services the request closest to the current head position next. Reduces average seek time but can cause starvation of far-away requests.
- SCAN (Elevator Algorithm): The head moves in one direction, servicing requests along the way, until it reaches the end of the disk, then reverses direction.
- C-SCAN (Circular SCAN): Like SCAN, but after reaching one end, the head jumps directly back to the beginning without servicing requests on the return trip, then continues in the same direction. This gives more uniform wait times.

File Allocation Methods:

- Contiguous Allocation: Each file occupies a set of contiguous blocks. Fast access but suffers from external fragmentation.
- Linked Allocation: Each file is a linked list of blocks scattered anywhere on disk. No external fragmentation, but slow random access.
- Indexed Allocation: Each file has an index block containing pointers to all its data blocks. Supports fast direct access without fragmentation issues.

Program 1: FCFS Disk Scheduling

To service disk requests in the order they arrive and calculate total head movement.

Program code

```
#include <stdio.h>
int main()
{
   int n, i, head, seek = 0, req[50];
   printf("Enter number of requests: ");
   scanf("%d", &n);
   printf("Enter request sequence: ");
   for(i = 0; i < n; i++)
      scanf("%d", &req[i]);
   printf("Enter initial head position: ");
   scanf("%d", &head);

   for(i = 0; i < n; i++)
   {
      seek += abs(req[i] - head);
      head = req[i];
   }
   printf("\nTotal Head Movement = %d\n", seek);
   printf("Average Seek Time = %.2f\n", (float)seek / n);
   return 0;
}
```

Compile: gcc fcfs.c -o fcfs

Run: ./fcfs

Sample Input: Requests = 82, 170, 43, 140, 24, 16, 190; Initial head = 50

Output

```
Enter number of requests: 7
Enter request sequence: 82 170 43 140 24 16 190
Enter initial head position: 50

Total Head Movement = 642
Average Seek Time = 91.71
```

Program 2: SSTF Disk Scheduling

To service the nearest pending request first and minimize seek time.

Program code

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int n, i, j, head, seek = 0, req[50], visited[50] = {0};
   printf("Enter number of requests: ");
   scanf("%d", &n);
   printf("Enter request sequence: ");
   for(i = 0; i < n; i++)
      scanf("%d", &req[i]);
   printf("Enter initial head position: ");
   scanf("%d", &head);

   for(i = 0; i < n; i++)
   {
      int minDist = 99999, idx = -1;
      for(j = 0; j < n; j++)
      {
         if(!visited[j] && abs(req[j] - head) < minDist)
         {
            minDist = abs(req[j] - head);
            idx = j;
         }
      }
      seek += minDist;
      head = req[idx];
      visited[idx] = 1;
   }
   printf("\nTotal Head Movement = %d\n", seek);
   printf("Average Seek Time = %.2f\n", (float)seek / n);
   return 0;
}
```

Compile: gcc sstf.c -o sstf

Run: ./sstf

Sample Input: Same as above (Requests = 82, 170, 43, 140, 24, 16, 190; head = 50)

Output

```
Enter number of requests: 7
Enter request sequence: 82 170 43 140 24 16 190
```

```
Enter initial head position: 50

Total Head Movement = 208
Average Seek Time = 29.71
```

Program 3: SCAN Disk Scheduling

To service requests by moving in one direction until the disk end, then reversing.

Program code

```
#include <stdio.h>
#include <stdlib.h>
#define DISK_SIZE 200
int cmp(const void *a, const void *b) { return (*(int*)a - *(int*)b); }
int main()
{
   int n, i, head, seek = 0, req[50];
   printf("Enter number of requests: ");
   scanf("%d", &n);
   printf("Enter request sequence: ");
   for(i = 0; i < n; i++)
      scanf("%d", &req[i]);
   printf("Enter initial head position: ");
   scanf("%d", &head);

   qsort(req, n, sizeof(int), cmp);

   int index = 0;
   while(index < n && req[index] < head) index++;

   /* service upward to end of disk */
   int pos = head;
   for(i = index; i < n; i++)
   {
      seek += abs(req[i] - pos);
      pos = req[i];
   }
   seek += abs((DISK_SIZE - 1) - pos);
   pos = DISK_SIZE - 1;

   /* reverse and service remaining */
   for(i = index - 1; i >= 0; i--)
   {
      seek += abs(pos - req[i]);
      pos = req[i];
```

```
    }

    printf("\nTotal Head Movement = %d\n", seek);
    printf("Average Seek Time = %.2f\n", (float)seek / n);
    return 0;
}
```

Compile: gcc scan.c -o scan

Run: ./scan

Sample Input: Requests = 82, 170, 43, 140, 24, 16, 190; head = 50; Disk size = 200 (0–199)

Output

```
Enter number of requests: 7
Enter request sequence: 82 170 43 140 24 16 190
Enter initial head position: 50

Total Head Movement = 332
Average Seek Time = 47.43
```

Program 4: C-SCAN Disk Scheduling

To service requests in one direction, jump to the opposite end, and continue in the same direction.

Program code

```
#include <stdio.h>
#include <stdlib.h>
#define DISK_SIZE 200
int cmp(const void *a, const void *b) { return (*(int*)a - *(int*)b); }
int main()
{
    int n, i, head, seek = 0, req[50];
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request sequence: ");
    for(i = 0; i < n; i++)
        scanf("%d", &req[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);

    qsort(req, n, sizeof(int), cmp);

    int index = 0;
    while(index < n && req[index] < head) index++;

    int pos = head;
```

```
   /* service upward to end of disk */
   for(i = index; i < n; i++)
   {
      seek += abs(req[i] - pos);
      pos = req[i];
   }
   seek += abs((DISK_SIZE - 1) - pos);

   /* jump to beginning of disk */
   seek += (DISK_SIZE - 1);
   pos = 0;

   /* service remaining requests from start */
   for(i = 0; i < index; i++)
   {
      seek += abs(req[i] - pos);
      pos = req[i];
   }

   printf("\nTotal Head Movement = %d\n", seek);
   printf("Average Seek Time = %.2f\n", (float)seek / n);
   return 0;
}
```

Compile: gcc cscan.c -o cscan

Run: ./cscan

Sample Input: Requests = 82, 170, 43, 140, 24, 16, 190; head = 50; Disk size = 200 (0–199)

Output

```
Enter number of requests: 7
Enter request sequence: 82 170 43 140 24 16 190
Enter initial head position: 50

Total Head Movement = 391
Average Seek Time = 55.86
```

Program 5: Contiguous File Allocation Simulation

To simulate contiguous file allocation and calculate storage utilization.

Program code

```
#include <stdio.h>
#define DISK_BLOCKS 50
int main()
{
```

```
    int disk[DISK_BLOCKS] = {0};
    int start, len, i, files, usedBlocks = 0;
    printf("Enter number of files: ");
    scanf("%d", &files);

    for(i = 0; i < files; i++)
    {
        printf("Enter start block and length for file %d: ", i + 1);
        scanf("%d %d", &start, &len);
        for(int j = start; j < start + len && j < DISK_BLOCKS; j++)
        {
            disk[j] = i + 1;
            usedBlocks++;
        }
    }

    printf("\nDisk Block Allocation Map:\n");
    for(i = 0; i < DISK_BLOCKS; i++)
        printf("%d ", disk[i]);

    printf("\n\nUsed Blocks = %d / %d\n", usedBlocks, DISK_BLOCKS);
    printf("Storage Utilization = %.2f%%\n", (float)usedBlocks * 100 / DISK_BLOCKS);
    return 0;
}
```

Compile: gcc fileAlloc.c -o fileAlloc

Run: ./fileAlloc

Sample Input: 2 files — File 1: start=0, length=5; File 2: start=10, length=8

Output

```
Enter number of files: 2
Enter start block and length for file 1: 0 5
Enter start block and length for file 2: 10 8

Disk Block Allocation Map:
1 1 1 1 1 0 0 0 0 0 2 2 2 2 2 2 2 2 0 0 ...

Used Blocks = 13 / 50
Storage Utilization = 26.00%
```

Observation Table (Disk Scheduling Comparison)

Algorithm	Total Head Movement	Average Seek Time
FCFS	642	91.71
SSTF	208	29.71
SCAN	332	47.43
C-SCAN	391	55.86

(Requests: 82, 170, 43, 140, 24, 16, 190; Initial head = 50; Disk size = 200 tracks)

Important Formulas

Parameter	Formula
Total Head Movement	Sum of |current position - next request| over all moves
Average Seek Time	Total Head Movement / Number of Requests
Throughput	Number of Requests Serviced / Total Time Taken
Storage Utilization	(Used Blocks / Total Disk Blocks) x 100

Conclusion

Disk scheduling algorithms (FCFS, SSTF, SCAN, C-SCAN) were implemented and compared based on total head movement and average seek time. SSTF gave the lowest seek time for this request set but risks starvation, while SCAN and C-SCAN offer more balanced and predictable service. Contiguous file allocation was also simulated to observe its effect on storage utilization and fragmentation.

Viva Questions

1. What is disk scheduling and why is it necessary?
2. Compare FCFS, SSTF, SCAN, and C-SCAN in terms of seek time and fairness.
3. Why does C-SCAN provide more uniform waiting time than SCAN?
4. What is starvation in SSTF, and how do SCAN/C-SCAN avoid it?
5. Differentiate between contiguous, linked, and indexed file allocation methods.
6. How is storage utilization affected by fragmentation in contiguous allocation?

EXPERIMENT NO. 10

Title: Containerization and Resource Monitoring for AI/ML Workloads

Aim

To deploy a simple AI/ML application inside a Docker container, monitor CPU, memory, storage, and process utilization using Linux tools, and evaluate the benefits of container-based deployment.

CO Mapping

CO1, CO2, CO5

Objectives

- To understand the concept of containerization and how it differs from virtualization.
- To build a Docker image for a simple AI/ML application.
- To deploy and run the AI/ML application inside a Docker container.
- To monitor CPU, memory, storage, and process utilization of the running container using Linux and Docker tools.
- To evaluate the benefits of container-based deployment over traditional deployment.

Software Requirements

- Ubuntu Linux / WSL
- Docker Engine
- Python 3 with scikit-learn, pandas, numpy
- VS Code
- Linux monitoring tools (top, htop, docker stats, ps, du)

Theory

Containerization is a lightweight form of virtualization that packages an application together with its dependencies, libraries, and runtime environment into a single portable unit called a container. Unlike a virtual machine, a container shares the host operating system's kernel, which makes it faster to start, smaller in size, and more efficient in resource usage.

Docker is the most widely used containerization platform. A Docker image is a read-only template containing the application code and its dependencies, and a Docker container is a running instance of that image. Images are built using instructions written in a Dockerfile.

AI/ML workloads often depend on specific versions of libraries such as scikit-learn, TensorFlow, or PyTorch. Containerization solves the classic "it works on my machine" problem by bundling the exact environment the model was trained and tested in, making deployment consistent across development, testing, and production systems.

Benefits of Container-Based Deployment:

- Portability: the same container image runs identically on any host with Docker installed.

- Isolation: each container has its own filesystem, process space, and network namespace, preventing dependency conflicts.
- Efficient resource usage: containers share the host kernel, so they consume far less CPU, memory, and storage than full virtual machines.
- Fast startup: containers start in seconds since there is no separate OS to boot.
- Scalability: multiple containers of the same image can be started or stopped quickly to handle varying load.

Resource monitoring tools such as docker stats, top, htop, ps, and du allow an administrator to observe how much CPU, memory, storage, and how many processes a running container is actually using, which is essential for capacity planning and performance tuning of AI/ML deployments.

Program 1: Simple AI/ML Application (Python)

A simple machine learning script that trains a classifier on the Iris dataset and prints its accuracy. This is the application that will be containerized.

File: app.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import time

def main():
    data = load_iris()
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.2, random_state=42)

    model = RandomForestClassifier(n_estimators=100)
    model.fit(X_train, y_train)

    predictions = model.predict(X_test)
    acc = accuracy_score(y_test, predictions)

    print(f"Model trained successfully.")
    print(f"Test Accuracy = {acc * 100:.2f}%")

    # keep the process alive briefly so resource usage can be observed
    time.sleep(30)

if __name__ == "__main__":
    main()
```

File: requirements.txt

```
scikit-learn
pandas
numpy
```

Program 2: Dockerfile for the AI/ML Application

To containerize the Python ML application by defining its environment and dependencies in a Dockerfile.

File: Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

CMD ["python", "app.py"]
```

Build and Run Commands

Build image: docker build -t ml-app:1.0 .

Run container: docker run --name ml-container ml-app:1.0

List running containers: docker ps

List images: docker images

Output

```
$ docker build -t ml-app:1.0 .
[+] Building 12.4s (10/10) FINISHED
 => [1/4] FROM python:3.10-slim
 => [2/4] WORKDIR /app
 => [3/4] RUN pip install --no-cache-dir -r requirements.txt
 => [4/4] COPY app.py .
 => exporting to image
Successfully tagged ml-app:1.0

$ docker run --name ml-container ml-app:1.0
Model trained successfully.
Test Accuracy = 96.67%
```

Program 3: Monitoring Container Resource Utilization

To monitor CPU, memory, storage, and process utilization of the running container using Docker and Linux tools.

Command 1: Live CPU and Memory Usage

Command: docker stats ml-container

```
CONTAINER ID   NAME          CPU %    MEM USAGE / LIMIT     MEM %   NET I/O
BLOCK I/O
a1b2c3d4e5f6   ml-container   38.42%   142.3MiB / 1.943GiB   7.15%   1.2kB / 0B      0B / 0B
```

Command 2: Process List Inside the Container

Command: docker top ml-container

```
UID    PID    PPID   C   STIME   TTY   TIME       CMD
root   4821   4801   14  10:02   ?     00:00:02   python app.py
```

Command 3: Image and Container Storage Usage

Command: docker system df

```
TYPE          TOTAL   ACTIVE   SIZE     RECLAIMABLE
Images        1       1        152MB    0B (0%)
Containers    1       1        0B       0B
Local Volumes 0       0        0B       0B
```

Command 4: Host-Level Monitoring (while container runs)

Command: top (or htop)

```
PID   USER   PR  NI  VIRT   RES   SHR   S  %CPU  %MEM   TIME+    COMMAND
4821  root   20  0   412m   142m  38m   S  38.4  7.1    0:02.10  python
```

Observation Table

Parameter	**Native (Host) Execution**	**Docker Container Execution**
CPU Utilization	~40%	~38%
Memory Usage	~150 MB	~142 MB
Storage Footprint	System-wide install	~152 MB (image only)
Startup Time	~1 s	~1-2 s
Process Isolation	None	Yes (own namespace)

Note: CPU and memory overhead of containerization is minimal because containers share the host kernel rather than emulating separate hardware, unlike full virtual machines.

Conclusion

A simple AI/ML application was successfully containerized using Docker by defining its dependencies and runtime in a Dockerfile, building an image, and running it as an isolated container. Resource monitoring using docker stats, docker top, docker system df, and host-level tools such as top showed that the container's CPU and memory utilization closely matched native execution, with only a small and acceptable overhead. This confirms that container-based deployment offers portability, isolation, and efficient resource usage, making it well suited for deploying AI/ML workloads consistently across different environments.

Viva Questions

1. What is containerization, and how does it differ from traditional virtualization using VMs?
2. Explain the role of a Dockerfile, a Docker image, and a Docker container.
3. Why is Docker particularly useful for deploying AI/ML applications?
4. List and explain the Linux/Docker commands used to monitor CPU, memory, storage, and process utilization of a container.
5. What are the key benefits of container-based deployment over running an application directly on the host?
6. Compare the resource overhead of a Docker container with that of a full virtual machine.